BFOR516 - Transformers and LLMs

Srishti Gupta, Ph.D.

Encoder Decoder Architecture

Encoder-Decoder Model

Fixed-length representation → Even though LSTMs improve memory, they still summarize the entire input into a single hidden state vector, making it hard to handle very long sequences. This led to the encoder-decoder architecture.

The encoder-decoder model is a type of neural network architecture commonly used in tasks that require sequence-to-sequence (Seq2Seq) learning, such as machine translation, text summarization, and image captioning. It consists of two main components:

1. **Encoder**
    a. The encoder processes the input sequence (e.g., a sentence) and compresses it into a fixed-size representation (also called a context vector or latent space).
    b. It typically consists of a series of recurrent neural networks (RNNs), long short-term memory (LSTMs), gated recurrent units (GRUs), or transformers.
    c. The encoder reads the input sequentially and captures contextual information.

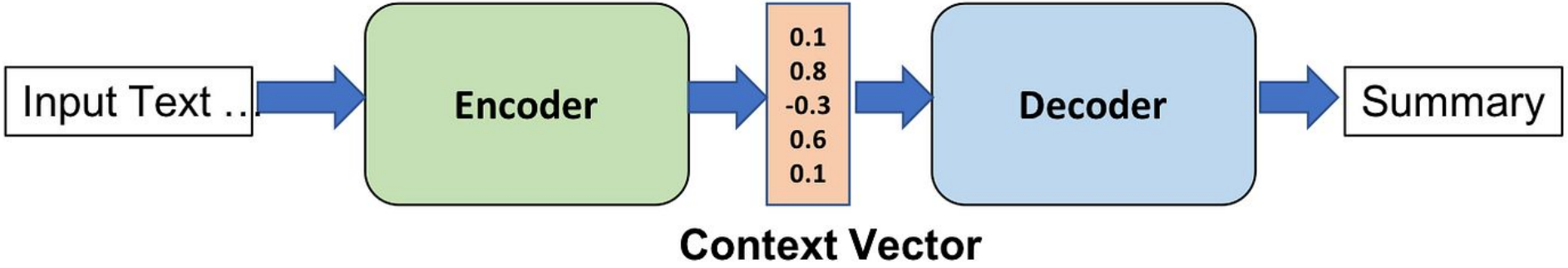
Encoder-Decoder Model

2. **Decoder**
   a. The decoder takes the context vector from the encoder and generates an output sequence.
   b. Like the encoder, it can use RNNs, LSTMs, GRUs, or transformers.
   c. The decoder generates one token at a time, using previously generated tokens as input (autoregressive behavior).
   d. In tasks like machine translation, the decoder uses a start-of-sequence (<SOS>) token to begin generating the output and stops when an end-of-sequence (<EOS>) token is reached.

How It Works

1. The input sequence is fed into the **encoder**, which generates a hidden representation.
2. This representation is passed to the **decoder**, which generates the output sequence step by step.

Encoder-Decoder Model



Multi-Head Attention

Multi-Head Attention

- Multi-head attention splits the input into multiple smaller parts (**heads**) and looks at the relationships in different ways at the same time.
- This helps in capturing different types of relationships—like short-term and long-term dependencies—within a sequence of words or data.

Why Do We Need Multi-Head Attention?

- In a regular attention mechanism, each word (or data point) compares itself with others in the input sequence to determine which ones are most relevant. However, relying on a single attention mechanism may not be enough because:
  - Some relationships are local (e.g., in a sentence, "New York is a city," "New York" and "city" are closely related).
  - Some relationships are global (e.g., in a paragraph, "New York" might relate to "USA" appearing much later in the text).
- Multi-head attention solves this by using multiple different attention mechanisms at the same time. Each "head" captures different aspects of relationships in the data.

How Does Multi-Head Attention Work?

1. Inputs: Queries, Keys, and Values
   a. Queries (Q): The current word/data point that needs attention.
   b. Keys (K): The other words/data points used for comparison.
   c. Values (V): The actual data associated with each key.
2. Multiple Attention Heads
   a. Instead of using just one set of queries, keys, and values, **we split them into multiple smaller sets.**
   b. Each set is processed independently using separate attention mechanisms.
   c. This allows each head to learn different types of relationships.
3. Combine the Heads
   a. The outputs from all the heads are **concatenated** (combined) and passed through a final transformation.
   b. This creates a richer, more informative final output.

Mathematical Explanation

Each head in multi-head attention computes its output as:

$$\text{Head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where:

- W_i^Q, W_i^K, W_i^V are learnable weight matrices that transform queries, keys, and values into different subspaces.
- The attention function used is usually **scaled dot-product attention**.

The final output is:

$$\text{MultiHead}(Q, K, V) = W^O(\text{Concat}(\text{Head}_1, ..., \text{Head}_h))$$

Where:

- W^O is another learnable weight matrix to transform the concatenated output.

Self-Attention and Positional Encoding

Self-Attention

Imagine you are reading a sentence, and you want to understand the meaning of a word. You naturally look at other words in the sentence to get the context. This is self-attention—each word (or token) in a sequence pays attention to all other words to understand their relationships.

For each token in a sequence, we compute three vectors:

- **Query (Q):** What am I looking for?
- **Key (K):** What do I offer?
- **Value (V):** What information do I carry?

Each token compares its Query to the Keys of all other tokens, and based on the similarity score, it decides how much attention to give each token. This results in a new representation of each token based on all the others.

Self-Attention

Mathematically, this is computed using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of the key vectors (used for scaling).

Positional Encoding

Unlike RNNs, which naturally process tokens in order, self-attention ignores sequence order. This is a problem because word order matters in a sentence.

Example:

- "The cat chased the mouse."
- "The mouse chased the cat."

Without knowing the order, the meaning is lost!

To solve this, we add positional encodings—extra numbers added to each token's representation that help the model distinguish positions.

We do this using sine and cosine functions:

$$PE_{\text{pos},2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

$$PE_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Where:

- pos = token position in the sequence
- d = dimension of embedding
- i = index of the embedding dimension

The Transformer Architecture

Introduction to Transformers

The Transformer architecture is a deep learning model designed primarily for tasks involving sequences of data, like language processing. Unlike earlier models that used Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs), the Transformer architecture relies purely on self-attention mechanisms. This offers a number of advantages, such as the ability to compute elements in parallel and handle very long input sequences efficiently.

- Self-attention allows each word (or token) in the input to "attend" to every other word in the sequence, considering their relationship.
- Parallel computation means that the model processes all tokens simultaneously, rather than sequentially (which is what RNNs do).

Transformers were first proposed for sequence-to-sequence tasks, like language translation, but they are now used in many areas including vision, speech, and reinforcement learning.

Structure of the Transformer Model

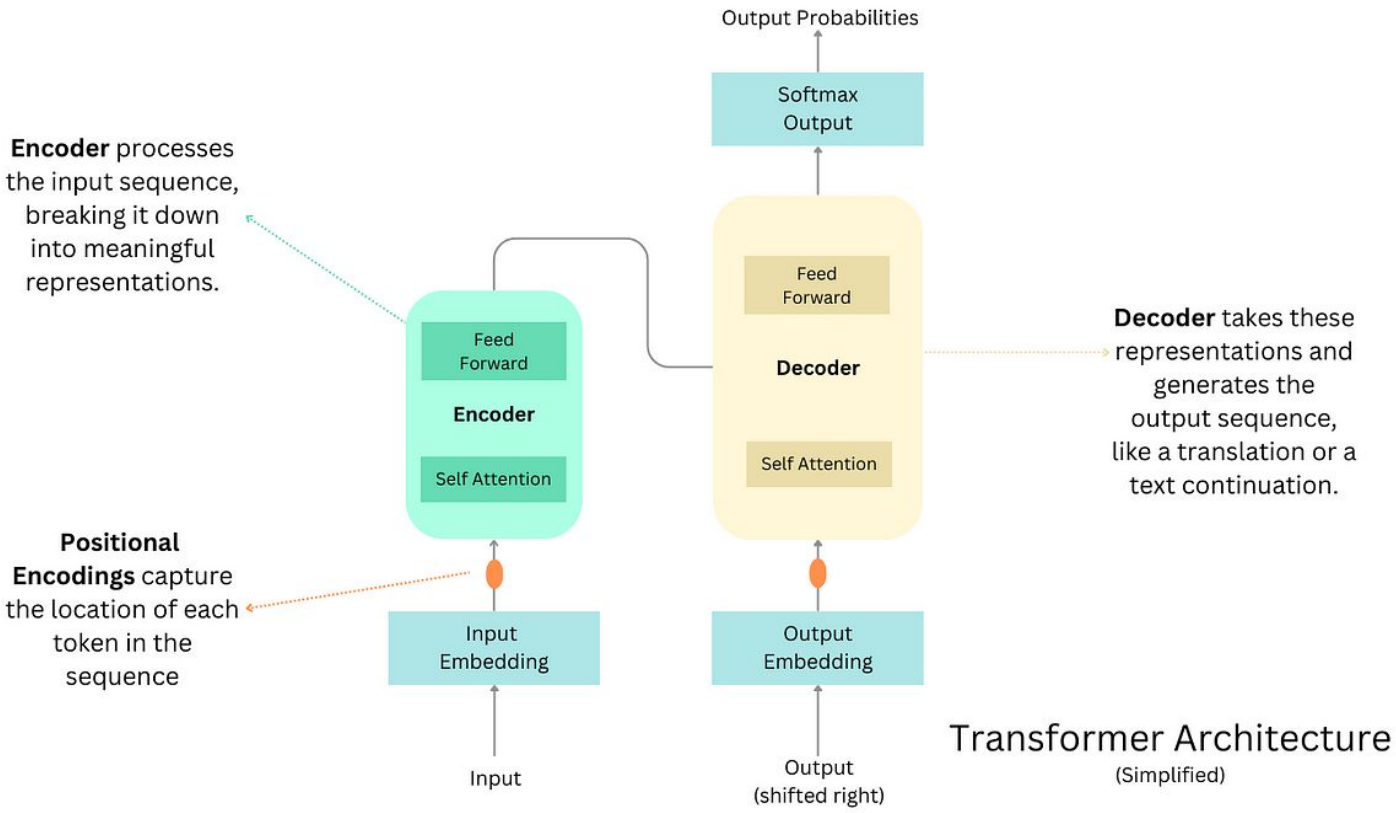
The Transformer follows an **encoder-decoder architecture**, where both parts consist of several identical layers. Below is a breakdown of the Transformer model:

1. **Encoder:**
   a. The encoder takes an input sequence (like a sentence) and processes it to create a **contextual representation** for each token in the sequence.
   b. It consists of multiple layers, each having two sub-layers:
      i. Multi-head self-attention: This allows each token to attend to all other tokens.
      ii. Position-wise feed-forward network: This applies the same transformation to every token independently.
   c. Each sub-layer in the encoder uses a residual connection followed by layer normalization.

Structure of the Transformer Model

2. **Decoder:**
   a. The decoder generates the output sequence (e.g., a translation) based on the context created by the encoder.
   b. Each decoder layer has three sub-layers:
      i. Masked self-attention: This ensures that predictions are made based on previously generated tokens (autoregressive property).
      ii. Encoder-decoder attention: This allows the decoder to focus on relevant parts of the encoder's output.
      iii. Position-wise feed-forward network: Same as in the encoder, applied to each token independently.

Structure of the Transformer Model



Transformer Architecture
(Simplified)

Step-by-Step Example with the Transformer

1. Positional Encoding

Transformers process words in parallel, so they don't inherently know the order of words (like RNNs do). To solve this, we add positional encoding to the input words. This encoding lets the model know the position of each word in the sequence.

For example:

- "I" -> position 1
- "love" -> position 2
- "programming" -> position 3

We represent these positions with numerical values (learned by the model).

Step-by-Step Example with the Transformer

2. **Encoder (Understanding Input)**

   Now, we feed the sequence **"I love programming"** into the encoder. The encoder has multiple layers, and each layer does two things:

   a. **Self-Attention**: The model looks at each word and decides how much attention it should give to every other word in the sentence. This is where the term "self-attention" comes from.
      i. For instance, the word **"love"** might have a higher attention to **"programming"** because they are more closely related.
      ii. The self-attention process is based on **queries, keys, and values**. Each word gets transformed into these three vectors, and the model calculates how much each word should attend to every other word.
   b. **Feed-Forward Network**: After attention, the output is passed through a **feed-forward neural network** that transforms the data before sending it to the next layer.

   Each encoder layer outputs a **vector** that represents the entire sequence, but with more meaningful information about relationships between the words.

Step-by-Step Example with the Transformer

3. Decoder (Generating Output)

   The decoder works similarly to the encoder, but it has a third important layer:

   a. Masked Self-Attention: When the decoder generates words, it can't see future words (because it's generating them step-by-step). This is where masking comes in: it ensures the model only looks at the previously generated words (not future ones).
   b. Encoder-Decoder Attention: This layer helps the decoder focus on the relevant parts of the input sequence (processed by the encoder). For example, when generating the French word for "love", it will pay more attention to the word "love" in the input sentence.
   c. Feed-Forward Network: Similar to the encoder, the decoder also has a feed-forward network that transforms the data.

   At each decoder layer, the output is passed through self-attention, encoder-decoder attention, and then a feed-forward network.

Large-Scale Pretraining with Transformers

What is Large-Scale Pretraining?

- In the past, models like Transformers were trained on specific tasks using small datasets (like translating English to French or classifying images). These models would only be experts in those tasks and struggle with generalizing to new tasks.
- Large-scale pretraining involves training a model on huge datasets, so it becomes a more general-purpose model. This means it can perform well on multiple tasks without needing to be retrained for each new one. For example, a model pretrained on a massive amount of text can later be used for tasks like translation, summarization, or question answering, without starting from scratch.

Transformer Models and Their Scalability

- Transformers (the architecture we're focusing on) perform better when they're trained on a lot of data, and their performance improves when they have more parameters (the parts of the model that learn from data) and more computing power.
- Scaling behavior: The bigger the model (more parameters), the more data it trains on, and the more computing resources you use, the better the model performs. This is why Transformers have done so well, especially with multimodal data (data that includes different types of information, like text, images, and sounds).

Types of Transformers

Transformers can be trained in different configurations. These include:

1. **Encoder-Only Transformers (e.g., BERT)**

- What is it?: The encoder in a Transformer reads the input data and processes it into a set of representations.
- Example: BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only model. It was pretrained by feeding it text with missing words (e.g., “I love <MASK> car”) and asking it to predict the missing word.
- How it works: BERT learns bidirectional context (it looks at both the words before and after the missing word). This makes it powerful for tasks like sentiment analysis (deciding if a sentence is positive or negative).

Types of Transformers

2. **Encoder-Decoder Transformers (e.g., T5, BART)**

- What is it?: This architecture has both an encoder and a decoder. The encoder processes input data, and the decoder generates output (e.g., translating text).
- Example: T5 (Text-to-Text Transfer Transformer) treats all tasks as text-to-text problems (like “summarize this article” or “translate this text to French”).
- How it works: The model is pretrained to take text (e.g., a sentence) and "corrupt" it (replace some words), then try to predict the original text. Once it’s pretrained, it can be fine-tuned to perform specific tasks like summarizing articles or answering questions.
- Fine-Tuning: T5 is trained on a huge amount of data (e.g., billions of tokens) and then fine-tuned on specific tasks like summarizing a news article.

Types of Transformers

3. Decoder-Only Transformers (e.g., GPT-3)

- What is it?: In this architecture, there is only a decoder that predicts text. It is typically used for language modeling (predicting the next word in a sentence).
- Example: GPT (Generative Pretrained Transformer) is a decoder-only model. It is pretrained to predict the next word in a sequence by looking at the previous words.
- How it works: During pretraining, GPT sees a sequence like “The cat is on the” and predicts the next word (“mat”). The model can generate entire paragraphs by repeatedly predicting the next word.
- Fine-Tuning: After pretraining, GPT can be fine-tuned for specific tasks like text completion or question answering.